

```
"""
```

```
SQLite storage layer for the stumble app's content pool.
```

```
Two tables:
```

```
    sources -- the RSS feeds / sites we pull from
```

```
    items   -- individual stumble-able links, deduped by url_hash
```

```
Deliberately simple (no ORM) so it's easy to swap for Postgres later --  
just replace the connect() call and the ? placeholders with %s.
```

```
"""
```

```
import sqlite3
```

```
import hashlib
```

```
from contextlib import contextmanager
```

```
DB_PATH = "stumble.db"
```

```
SCHEMA = """
```

```
CREATE TABLE IF NOT EXISTS sources (
```

```
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    name          TEXT NOT NULL,
```

```
    site_url      TEXT NOT NULL,
```

```
    feed_url      TEXT NOT NULL UNIQUE,
```

```
    category      TEXT NOT NULL,
```

```
    active        INTEGER NOT NULL DEFAULT 1
```

```
);
```

```
CREATE TABLE IF NOT EXISTS items (
```

```
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    source_id     INTEGER NOT NULL REFERENCES sources(id),
```

```
    url           TEXT NOT NULL,
```

```
    url_hash      TEXT NOT NULL UNIQUE,
```

```
    title         TEXT NOT NULL,
```

```
    excerpt       TEXT,
```

```
    image_url     TEXT,
```

```
    tags          TEXT NOT NULL,           -- comma-separated, e.g. "design,craft"
```

```
    published_at  TEXT,
```

```
    ingested_at   TEXT NOT NULL DEFAULT (datetime('now')),
```

```
    quality_score INTEGER NOT NULL DEFAULT 0, -- bumped by manual curation later
```

```
    is_dead       INTEGER NOT NULL DEFAULT 0 -- flip on when link-checker finds
```

```
);
```

```
CREATE INDEX IF NOT EXISTS idx_items_tags ON items(tags);
```

```
CREATE INDEX IF NOT EXISTS idx_items_dead ON items(is_dead);
```

```
"""
```

```
def url_hash(url: str) -> str:
    """Stable dedupe key -- strips nothing fancy, just normalizes trailing slash.
    normalized = url.strip().rstrip("/")
    return hashlib.sha256(normalized.encode("utf-8")).hexdigest()
```

```
@contextmanager
def get_conn(path: str = DB_PATH):
    conn = sqlite3.connect(path)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
```

```
def init_db(path: str = DB_PATH):
    with get_conn(path) as conn:
        conn.executescript(SCHEMA)
```

```
def upsert_source(conn, name: str, site_url: str, feed_url: str, category: str) -
    cur = conn.execute(
        """INSERT INTO sources (name, site_url, feed_url, category)
        VALUES (?, ?, ?, ?)
        ON CONFLICT(feed_url) DO UPDATE SET name=excluded.name, category=exclu
        RETURNING id""",
        (name, site_url, feed_url, category),
    )
    return cur.fetchone()["id"]
```

```
def insert_item(conn, source_id: int, url: str, title: str, excerpt: str,
                image_url: str, tags: list[str], published_at: str | None) -> bo
    """Returns True if inserted, False if it was a duplicate (already seen)."""
    h = url_hash(url)
    try:
        conn.execute(
            """INSERT INTO items
            (source_id, url, url_hash, title, excerpt, image_url, tags, publis
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)""",
            (source_id, url, h, title, excerpt, image_url, ",".join(tags), publis
        )
```

```
        return True
    except sqlite3.IntegrityError:
        return False # duplicate url_hash -- already in the pool
```